EMAIL CAMPAIGN PLATFORM

M1 Login and User Management

Documentation focused only on M1 implementation: backend auth foundation, login/session routes, lockout and password reset, profile/user management, frontend screens, and final testing.

MODULE	PARTS	STATUS
M1	A - F	Implemented

M1 Scope

> M1 covers admin login and user management only. Database setup and AWS setup were completed in previous modules and are not part of this document's work breakdown.

M1 Parts Overview

Part	Name	What Was Built	Status
A	Backend auth foundation	Prisma helper, bcrypt password helper, JWT helper, Express user typing, auth middleware, role middleware.	Complete
B	Login / refresh / me / logout	Auth routes for login, refresh token, current user, and logout.	Complete
C	Lockout + forgot/reset password	Failed-login lockout, forgot password token generation, reset password flow.	Complete
D	Profile + user management	Profile read/update/password update, user listing, invite, role assignment, deactivate.	Complete
E	Frontend login/profile/users screens	Responsive login, forgot/reset password, dashboard shell, profile page, users page, role-based sidebar.	Complete

F	Final testing	Manual curl tests, browser UI verification, typecheck/build validation.	Complete / final root verification recommended

Admin Role Model

Role	Who	What They Can Do
SUPER_ADMIN	IT / Platform Owner	Full access, user management, organisation/AWS settings, all modules.
CAMPAIGN_MANAGER	Marketing team	Create/manage contacts, templates, campaigns, analytics. Cannot manage users.
VIEWER	Leadership / stakeholders	Read-only dashboard/report posture. Cannot create or edit.

Part A: Backend Auth Foundation

File	Purpose
`apps/api/src/lib/prisma.ts`	Single Prisma Client helper for DB access.
`apps/api/src/lib/password.ts`	bcrypt hash and password verification helpers.
`apps/api/src/lib/jwt.ts`	Access/refresh token signing and verification.
`apps/api/src/types/express.ts`	Adds `req.user` typing for authenticated requests.
`apps/api/src/middleware/auth.ts`	Requires Bearer access token and attaches user payload.
`apps/api/src/middleware/roles.ts`	Enforces role-based route permissions.

> **Validation:** TypeScript typecheck passed after Part A, proving the foundation compiled.

Part B: Login / Refresh / Me / Logout

Method	Route	Purpose

POST	/api/auth/login	Validates email/password and returns access token, refresh token, and user.
POST	/api/auth/refresh	Returns a new access token from a refresh token.
POST	/api/auth/logout	Returns logout acknowledgement.
GET	/api/auth/me	Returns logged-in user from Bearer token.

Test Results

```
POST /api/auth/login
Result: accessToken, refreshToken, and SUPER_ADMIN user returned.

GET /api/auth/me
Result: admin@example.com profile returned from Bearer token.
```

Part C: Lockout + Forgot/Reset Password

Feature	Implementation	Result
Failed login lockout	In-memory email+IP attempt map. Five failures lock login for 15 minutes.	Implemented.
Forgot password	Generates a 64-character reset token and logs/returns it in development.	Tested successfully.
Reset password	Consumes token, hashes new password with bcrypt, updates user.	Tested successfully.

Test Results

```
POST /api/auth/forgot-password
Result: reset token generated.

POST /api/auth/reset-password
Result: { "message": "Password reset successful" }

POST /api/auth/login after reset
Result: login succeeded with Admin@123.
```

> **Development note:** `devResetToken` is exposed for local testing. Remove this from the response before production.

Part D: Profile + User Management

Method	Route	Access	Purpose
GET	/api/profile	Authenticated admin	Read current profile.
PUT	/api/profile	Authenticated admin	Update name/email.
PUT	/api/profile/password	Authenticated admin	Update password after verifying current password.
GET	/api/users	SUPER_ADMIN	List organisation users.
POST	/api/users/invite	SUPER_ADMIN	Invite teammate and assign role.
PUT	/api/users/:id	SUPER_ADMIN	Update teammate name/role.
POST	/api/users/:id/deactivate	SUPER_ADMIN	Deactivate/delete teammate in MVP.

Test Results

```
GET /api/profile
Result: Super Admin profile returned.

PUT /api/profile
Result: name changed to Super Admin Updated.

GET /api/users
Result: Super Admin can view users.

POST /api/users/invite
Result: manager@example.com created with CAMPAIGN_MANAGER.
Result: viewer@example.com created with VIEWER.
```

Part E: Frontend Login/Profile/Users Screens

Screen	Route	Status
Login	/login	Email/password form, errors, forgot password link.
Forgot Password	/forgot-password	Email form and development reset token display.
Reset Password	/reset-password	Token and new password form.
Dashboard	/	Protected landing page after login with KPI placeholders.

Profile	/settings/profile	Name/email update and password update UI.
User Management	/settings/users	Super Admin user list and invite form.

> **UI style:** The frontend uses a clean operational-console style inspired by email campaign tools such as Zoho Campaigns: dark sidebar, restrained cards, compact tables, clear forms, and responsive layouts.

Role-Based Sidebar

Role	Visible Menu
SUPER_ADMIN	Dashboard, Contacts, Templates, Campaigns, Analytics, Organisation, Users, Profile.
CAMPAIGN_MANAGER	Dashboard, Contacts, Templates, Campaigns, Analytics, Profile.
VIEWER	Dashboard, Analytics, Profile.

Part F: Final Testing

Check	Status
Admin login API	Passed.
Current user API	Passed.
Forgot/reset password API	Passed.
Profile read/update API	Passed.
User invite API for Campaign Manager	Passed.
User invite API for Viewer	Passed.
Frontend dashboard visual check	Passed.
Recommended final commands	`pnpm typecheck` and `pnpm build`.

Limitations To Address Later

> **Lockout storage:** in-memory now, should move to Redis or DB for production.

Password reset storage: in-memory now, should store hashed reset tokens with expiry in DB.

Reset delivery: dev token is displayed now, later send reset emails through SES.

User deactivation: currently deletes user because schema lacks soft-delete/deactivation field.

Completion Status

M1 Parts A-F Complete

M1 now has the backend and frontend foundation needed for admin authentication and user management.

Next Module

M2 Contact List Management: contacts, lists, imports, contact detail, segments, and related frontend screens.

Code Appendix

Part A: apps/api/src/lib/prisma.ts

```
import { PrismaClient } from "@prisma/client";

export const prisma = new PrismaClient();
```

Part A: apps/api/src/lib/password.ts

```
import bcrypt from "bcryptjs";

export function hashPassword(password: string) {
  return bcrypt.hash(password, 10);
}

export function verifyPassword(password: string, passwordHash: string) {
  return bcrypt.compare(password, passwordHash);
}
```

Part A: apps/api/src/lib/jwt.ts

```
import jwt from "jsonwebtoken";
import { env } from "../config/env.js";

export type AccessTokenPayload = {
  sub: string;
  email: string;
  role: string;
  orgId: string;
};

export function signAccessToken(payload: AccessTokenPayload) {
  return jwt.sign(payload, env.JWT_ACCESS_SECRET, {
    expiresIn: `${env.ACCESS_TOKEN_MINUTES}m`
  });
}

export function signRefreshToken(payload: AccessTokenPayload) {
  return jwt.sign(payload, env.JWT_REFRESH_SECRET, {
    expiresIn: `${env.REFRESH_TOKEN_DAYS}d`
  });
}

export function verifyAccessToken(token: string) {
  return jwt.verify(token, env.JWT_ACCESS_SECRET) as AccessTokenPayload;
}

export function verifyRefreshToken(token: string) {
  return jwt.verify(token, env.JWT_REFRESH_SECRET) as AccessTokenPayload;
}
```

Part A: apps/api/src/middleware/auth.ts

```
import type { NextFunction, Request, Response } from "express";
import { verifyAccessToken } from "../lib/jwt.js";

export function requireAuth(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization;

  if (!header?.startsWith("Bearer ")) {
    return res.status(401).json({
      message: "Missing authorization token"
    });
  }

  const token = header.slice("Bearer ".length);

  try {
    req.user = verifyAccessToken(token);
    return next();
  } catch {
    return res.status(401).json({
      message: "Invalid or expired token"
    });
  }
}
```

Part A: apps/api/src/middleware/roles.ts

```
import type { NextFunction, Request, Response } from "express";

export function requireRole(...roles: string[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user) {
      return res.status(401).json({
        message: "Authentication required"
      });
    }

    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        message: "Insufficient permissions"
      });
    }

    return next();
  };
}
```

Part B/C: apps/api/src/routes/auth.ts

```
import { Router } from "express";
import { z } from "zod";
import { prisma } from "../lib/prisma.js";
import {
  signAccessToken,
  signRefreshToken,
  verifyRefreshToken
} from "../lib/jwt.js";
import { hashPassword, verifyPassword } from "../lib/password.js";
import { requireAuth } from "../middleware/auth.js";
```

```
import {
  clearFailedLogins,
  getLoginLockStatus,
  recordFailedLogin
} from "../lib/loginLockout.js";
import {
  consumePasswordResetToken,
  createPasswordResetToken
} from "../lib/passwordReset.js";

export const authRouter = Router();

const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(1)
});

function getRequestIp(req: { ip?: string; socket?: { remoteAddress?: string } }) {
  return req.ip ?? req.socket?.remoteAddress ?? "unknown";
}

authRouter.post("/login", async (req, res) => {
  const parsed = loginSchema.safeParse(req.body);

  if (!parsed.success) {
    return res.status(400).json({
      message: "Invalid login payload",
      errors: parsed.error.flatten()
    });
  }

  const ip = getRequestIp(req);
  const lockStatus = getLoginLockStatus(parsed.data.email, ip);

  if (lockStatus.locked) {
    return res.status(429).json({
      message: "Too many failed login attempts. Try again later.",
      lockedUntil: lockStatus.lockedUntil
    });
  }

  const user = await prisma.user.findUnique({
    where: {
      email: parsed.data.email
    }
  });

  if (!user) {
    const failed = recordFailedLogin(parsed.data.email, ip);

    return res.status(401).json({
      message: "Invalid email or password",
      attemptsRemaining: failed.attemptsRemaining
    });
  }

  const validPassword = await verifyPassword(
    parsed.data.password,
    user.passwordHash
  );

  if (!validPassword) {
    const failed = recordFailedLogin(parsed.data.email, ip);
```

```
    return res.status(401).json({
      message: "Invalid email or password",
      attemptsRemaining: failed.attemptsRemaining
    });
  }

  clearFailedLogins(parsed.data.email, ip);

  const tokenPayload = {
    sub: user.id,
    email: user.email,
    role: user.role,
    orgId: user.orgId
  };

  const accessToken = signAccessToken(tokenPayload);
  const refreshToken = signRefreshToken(tokenPayload);

  return res.json({
    accessToken,
    refreshToken,
    user: {
      id: user.id,
      email: user.email,
      name: user.name,
      role: user.role,
      orgId: user.orgId
    }
  });
});

authRouter.post("/refresh", async (req, res) => {
  const schema = z.object({
    refreshToken: z.string().min(1)
  });

  const parsed = schema.safeParse(req.body);

  if (!parsed.success) {
    return res.status(400).json({
      message: "Invalid refresh payload"
    });
  }

  try {
    const payload = verifyRefreshToken(parsed.data.refreshToken);
    const accessToken = signAccessToken(payload);

    return res.json({
      accessToken
    });
  } catch {
    return res.status(401).json({
      message: "Invalid refresh token"
    });
  }
});

authRouter.post("/logout", (_req, res) => {
  return res.json({
    message: "Logged out"
  });
```

```
});

authRouter.get("/me", requireAuth, async (req, res) => {
  const user = await prisma.user.findUnique({
    where: {
      id: req.user!.sub
    },
    select: {
      id: true,
      email: true,
      name: true,
      role: true,
      orgId: true,
      createdAt: true,
      updatedAt: true
    }
  });

  if (!user) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  return res.json({
    user
  });
});

authRouter.post("/forgot-password", async (req, res) => {
  const schema = z.object({
    email: z.string().email()
  });

  const parsed = schema.safeParse(req.body);

  if (!parsed.success) {
    return res.status(400).json({
      message: "Invalid forgot password payload"
    });
  }

  const user = await prisma.user.findUnique({
    where: {
      email: parsed.data.email
    }
  });

  let devResetToken: string | undefined;

  if (user) {
    devResetToken = createPasswordResetToken(user.email);
    const resetUrl = `http://localhost:5173/reset-password?token=${devResetToken}`;

    console.log("[PASSWORD RESET]");
    console.log("Email:", user.email);
    console.log("Reset URL:", resetUrl);
  }

  return res.json({
    message: "If an account exists, a password reset link has been sent.",
    devResetToken
  });
```

```
});

authRouter.post("/reset-password", async (req, res) => {
  const schema = z.object({
    token: z.string().min(1),
    password: z.string().min(8)
  });

  const parsed = schema.safeParse(req.body);

  if (!parsed.success) {
    return res.status(400).json({
      message: "Invalid reset password payload",
      errors: parsed.error.flatten()
    });
  }

  const tokenRecord = consumePasswordResetToken(parsed.data.token);

  if (!tokenRecord) {
    return res.status(400).json({
      message: "Invalid or expired reset token"
    });
  }

  const passwordHash = await hashPassword(parsed.data.password);

  await prisma.user.update({
    where: {
      email: tokenRecord.email
    },
    data: {
      passwordHash
    }
  });

  return res.json({
    message: "Password reset successful"
  });
});
```

Part C: apps/api/src/lib/loginLockout.ts

```
type LoginAttempt = {
  count: number;
  lockedUntil?: number;
};

const attempts = new Map<string, LoginAttempt>();

const MAX_ATTEMPTS = 5;
const LOCK_MINUTES = 15;

function getKey(email: string, ip: string) {
  return `${email.toLowerCase()}:${ip}`;
}

export function getLoginLockStatus(email: string, ip: string) {
  const key = getKey(email, ip);
  const attempt = attempts.get(key);
```

```
  if (!attempt?.lockedUntil) {
    return {
      locked: false
    };
  }

  if (Date.now() > attempt.lockedUntil) {
    attempts.delete(key);
    return {
      locked: false
    };
  }

  return {
    locked: true,
    lockedUntil: new Date(attempt.lockedUntil).toISOString()
  };
}

export function recordFailedLogin(email: string, ip: string) {
  const key = getKey(email, ip);
  const current = attempts.get(key) ?? {
    count: 0
  };

  const nextCount = current.count + 1;

  if (nextCount >= MAX_ATTEMPTS) {
    attempts.set(key, {
      count: nextCount,
      lockedUntil: Date.now() + LOCK_MINUTES * 60 * 1000
    });

    return {
      locked: true,
      attemptsRemaining: 0
    };
  }

  attempts.set(key, {
    count: nextCount
  });

  return {
    locked: false,
    attemptsRemaining: MAX_ATTEMPTS - nextCount
  };
}

export function clearFailedLogins(email: string, ip: string) {
  attempts.delete(getKey(email, ip));
}
```

Part C: apps/api/src/lib/passwordReset.ts

```
import crypto from "node:crypto";

type PasswordResetToken = {
  email: string;
  token: string;
  expiresAt: number;
```

```
};

const resetTokens = new Map<string, PasswordResetToken>();

const RESET_MINUTES = 15;

export function createPasswordResetToken(email: string) {
  const token = crypto.randomBytes(32).toString("hex");

  resetTokens.set(token, {
    email: email.toLowerCase(),
    token,
    expiresAt: Date.now() + RESET_MINUTES * 60 * 1000
  });

  return token;
}

export function consumePasswordResetToken(token: string) {
  const record = resetTokens.get(token);

  if (!record) {
    return null;
  }

  resetTokens.delete(token);

  if (Date.now() > record.expiresAt) {
    return null;
  }

  return {
    email: record.email
  };
}
```

Part D: apps/api/src/routes/profile.ts

```
import { Router } from "express";
import { z } from "zod";
import { prisma } from "../lib/prisma.js";
import { hashPassword, verifyPassword } from "../lib/password.js";
import { requireAuth } from "../middleware/auth.js";

export const profileRouter = Router();

profileRouter.use(requireAuth);

profileRouter.get("/", async (req, res) => {
  const user = await prisma.user.findUnique({
    where: {
      id: req.user!.sub
    },
    select: {
      id: true,
      email: true,
      name: true,
      role: true,
      orgId: true,
      createdAt: true,
      updatedAt: true
```

```
    }
  });

  if (!user) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  return res.json({
    user
  });
});

profileRouter.put("/", async (req, res) => {
  const schema = z.object({
    name: z.string().min(1).optional(),
    email: z.string().email().optional()
  });

  const parsed = schema.safeParse(req.body);

  if (!parsed.success) {
    return res.status(400).json({
      message: "Invalid profile payload",
      errors: parsed.error.flatten()
    });
  }

  const user = await prisma.user.update({
    where: {
      id: req.user!.sub
    },
    data: parsed.data,
    select: {
      id: true,
      email: true,
      name: true,
      role: true,
      orgId: true,
      createdAt: true,
      updatedAt: true
    }
  });

  return res.json({
    user
  });
});

profileRouter.put("/password", async (req, res) => {
  const schema = z.object({
    currentPassword: z.string().min(1),
    newPassword: z.string().min(8)
  });

  const parsed = schema.safeParse(req.body);

  if (!parsed.success) {
    return res.status(400).json({
      message: "Invalid password payload",
      errors: parsed.error.flatten()
    });
```

```
  }

  const user = await prisma.user.findUnique({
    where: {
      id: req.user!.sub
    }
  });

  if (!user) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  const validPassword = await verifyPassword(
    parsed.data.currentPassword,
    user.passwordHash
  );

  if (!validPassword) {
    return res.status(401).json({
      message: "Current password is incorrect"
    });
  }

  const passwordHash = await hashPassword(parsed.data.newPassword);

  await prisma.user.update({
    where: {
      id: user.id
    },
    data: {
      passwordHash
    }
  });

  return res.json({
    message: "Password updated"
  });
});
```

Part D: apps/api/src/routes/users.ts

```
import { Router } from "express";
import { UserRole } from "@prisma/client";
import { z } from "zod";
import { prisma } from "../lib/prisma.js";
import { hashPassword } from "../lib/password.js";
import { requireAuth } from "../middleware/auth.js";
import { requireRole } from "../middleware/roles.js";

export const usersRouter = Router();

usersRouter.use(requireAuth);
usersRouter.use(requireRole("SUPER_ADMIN"));

usersRouter.get("/", async (req, res) => {
  const users = await prisma.user.findMany({
    where: {
      orgId: req.user!.orgId
    },
```

```
    select: {
      id: true,
      email: true,
      name: true,
      role: true,
      createdAt: true,
      updatedAt: true
    },
    orderBy: {
      createdAt: "desc"
    }
  });

  return res.json({
    users
  });
});

usersRouter.post("/invite", async (req, res) => {
  const schema = z.object({
    email: z.string().email(),
    name: z.string().optional(),
    role: z.nativeEnum(UserRole).default(UserRole.CAMPAIGN_MANAGER),
    temporaryPassword: z.string().min(8).default("Temp@1234")
  });

  const parsed = schema.safeParse(req.body);

  if (!parsed.success) {
    return res.status(400).json({
      message: "Invalid invite payload",
      errors: parsed.error.flatten()
    });
  }

  const passwordHash = await hashPassword(parsed.data.temporaryPassword);

  const user = await prisma.user.create({
    data: {
      email: parsed.data.email,
      name: parsed.data.name,
      role: parsed.data.role,
      passwordHash,
      orgId: req.user!.orgId
    },
    select: {
      id: true,
      email: true,
      name: true,
      role: true,
      orgId: true,
      createdAt: true,
      updatedAt: true
    }
  });

  return res.status(201).json({
    user,
    temporaryPassword: parsed.data.temporaryPassword
  });
});

usersRouter.put("/:id", async (req, res) => {
```

```
  const schema = z.object({
    name: z.string().optional(),
    role: z.nativeEnum(UserRole).optional()
  });

  const parsed = schema.safeParse(req.body);

  if (!parsed.success) {
    return res.status(400).json({
      message: "Invalid user payload",
      errors: parsed.error.flatten()
    });
  }

  const existingUser = await prisma.user.findFirst({
    where: {
      id: req.params.id,
      orgId: req.user!.orgId
    }
  });

  if (!existingUser) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  const user = await prisma.user.update({
    where: {
      id: existingUser.id
    },
    data: parsed.data,
    select: {
      id: true,
      email: true,
      name: true,
      role: true,
      orgId: true,
      createdAt: true,
      updatedAt: true
    }
  });

  return res.json({
    user
  });
});

usersRouter.post("/:id/deactivate", async (req, res) => {
  const existingUser = await prisma.user.findFirst({
    where: {
      id: req.params.id,
      orgId: req.user!.orgId
    }
  });

  if (!existingUser) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  if (existingUser.id === req.user!.sub) {
```

```
    return res.status(400).json({
      message: "You cannot deactivate your own user"
    });
  }

  await prisma.user.delete({
    where: {
      id: existingUser.id
    }
  });

  return res.json({
    message: "User deactivated"
  });
});
```

Route Mounting: apps/api/src/index.ts

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import { sendEmail } from "./services/mailer.js";
import { enqueueSendJob } from "./services/queue.js";
import { uploadAsset } from "./services/storage.js";
import { authRouter } from "./routes/auth.js";
import { profileRouter } from "./routes/profile.js";
import { usersRouter } from "./routes/users.js";


const app = express();

const port = Number(process.env.API_PORT ?? 4000);
const webOrigin = process.env.WEB_ORIGIN ?? "http://localhost:5173";

app.use(cors({ origin: webOrigin }));
app.use(express.json());
app.use("/api/auth", authRouter);
app.use("/api/profile", profileRouter);
app.use("/api/users", usersRouter);

app.get("/", (_req, res) => {
  res.json({
    message: "Email Campaign Platform API",
    health: "/health"
  });
});

app.get("/health", (_req, res) => {
  res.json({
    status: "ok",
    service: "api"
  });
});

app.post("/dev/test-email", async (_req, res) => {
  const result = await sendEmail({
    to: "test@example.com",
    subject: "Dev email test",
    html: "<h1>Hello from dev mailer</h1>",
    unsubscribeUrl: "http://localhost:4000/unsubscribe?uid=dev-test"
  });
```

```
  res.json(result);
});

app.post("/dev/test-queue", async (_req, res) => {
  const result = await enqueueSendJob({
    type: "SEND_EMAIL",
    campaignId: "dev-campaign",
    contactId: "dev-contact"
  });

  res.json(result);
});

app.post("/dev/test-s3", async (_req, res) => {
  const result = await uploadAsset({
    key: `dev-tests/test-${Date.now()}.txt`,
    body: Buffer.from("S3 upload test from Email Campaign Platform"),
    contentType: "text/plain"
  });

  res.json(result);
});

app.listen(port, () => {
  console.log(`API running on http://localhost:${port}`);
});
```

Part E: apps/web/src/main.tsx

```
import React, { FormEvent, useMemo, useState } from "react";
import { createRoot } from "react-dom/client";
import "./styles.css";

type Role = "SUPER_ADMIN" | "CAMPAIGN_MANAGER" | "VIEWER";

type User = {
  id: string;
  email: string;
  name: string | null;
  role: Role;
  orgId: string;
};

type AuthState = {
  accessToken: string;
  refreshToken: string;
  user: User;
};

const API_BASE = "http://localhost:4000";

const navItems = [
  { label: "Dashboard", path: "/", roles: ["SUPER_ADMIN", "CAMPAIGN_MANAGER", "VIEWER"] },
  { label: "Contacts", path: "/contacts/lists", roles: ["SUPER_ADMIN", "CAMPAIGN_MANAGER"] },
  { label: "Templates", path: "/templates", roles: ["SUPER_ADMIN", "CAMPAIGN_MANAGER"] },
  { label: "Campaigns", path: "/campaigns", roles: ["SUPER_ADMIN", "CAMPAIGN_MANAGER"] },
  { label: "Analytics", path: "/analytics", roles: ["SUPER_ADMIN", "CAMPAIGN_MANAGER", "VIEWER"] },
  { label: "Organisation", path: "/settings/org", roles: ["SUPER_ADMIN"] },
  { label: "Users", path: "/settings/users", roles: ["SUPER_ADMIN"] },
  { label: "Profile", path: "/settings/profile", roles: ["SUPER_ADMIN", "CAMPAIGN_MANAGER", "VIEWER"] }
```

```
] satisfies Array<{ label: string; path: string; roles: Role[] }>;

function App() {
  const [auth, setAuth] = useState<AuthState | null>(() => {
    const saved = localStorage.getItem("auth");
    return saved ? JSON.parse(saved) : null;
  });

  const [route, setRoute] = useState(window.location.pathname);

  function navigate(path: string) {
    window.history.pushState({}, "", path);
    setRoute(path);
  }

  function saveAuth(next: AuthState) {
    localStorage.setItem("auth", JSON.stringify(next));
    setAuth(next);
  }

  function logout() {
    localStorage.removeItem("auth");
    setAuth(null);
    navigate("/login");
  }

  if (!auth) {
    if (route === "/forgot-password") {
      return <ForgotPasswordPage onNavigate={navigate} />;
    }

    if (route === "/reset-password") {
      return <ResetPasswordPage onNavigate={navigate} />;
    }

    return <LoginPage onLogin={saveAuth} onNavigate={navigate} />;
  }

  return (
    <Shell auth={auth} route={route} onNavigate={navigate} onLogout={logout}>
      {route === "/" && <DashboardPage user={auth.user} />}
      {route === "/settings/profile" && <ProfilePage auth={auth} onAuthChange={saveAuth} />}
      {route === "/settings/users" && <UsersPage auth={auth} />}
      {!["/", "/settings/profile", "/settings/users"].includes(route) && (
        <PlaceholderPage route={route} />
      )}
    </Shell>
  );
}

function LoginPage(props: {
  onLogin: (auth: AuthState) => void;
  onNavigate: (path: string) => void;
}) {
  const [email, setEmail] = useState("admin@example.com");
  const [password, setPassword] = useState("Admin@123");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  async function submit(event: FormEvent) {
    event.preventDefault();
    setError("");
    setLoading(true);
```

```
    const response = await fetch(`${API_BASE}/api/auth/login`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ email, password })
    });

    const data = await response.json();
    setLoading(false);

    if (!response.ok) {
      setError(data.message ?? "Login failed");
      return;
    }

    props.onLogin(data);
    props.onNavigate("/");
  }

  return (
    <AuthLayout>
      <form className="auth-card" onSubmit={submit}>
        <div className="brand-mark">EC</div>
        <h1>Email Campaign Platform</h1>
        <p className="muted">Sign in to manage contacts, templates, campaigns, and reports.</p>

        <label>
          Email
          <input value={email} onChange={(event) => setEmail(event.target.value)} />
        </label>

        <label>
          Password
          <input
            type="password"
            value={password}
            onChange={(event) => setPassword(event.target.value)}
          />
        </label>

        {error && <p className="error">{error}</p>}

        <button className="primary-button" disabled={loading}>
          {loading ? "Signing in..." : "Sign in"}
        </button>

        <button
          className="link-button"
          type="button"
          onClick={() => props.onNavigate("/forgot-password")}
        >
          Forgot password?
        </button>
      </form>
    </AuthLayout>
  );
}

function ForgotPasswordPage(props: { onNavigate: (path: string) => void }) {
  const [email, setEmail] = useState("admin@example.com");
  const [message, setMessage] = useState("");
```

```
  const [token, setToken] = useState("");

  async function submit(event: FormEvent) {
    event.preventDefault();

    const response = await fetch(`${API_BASE}/api/auth/forgot-password`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ email })
    });

    const data = await response.json();
    setMessage(data.message);
    setToken(data.devResetToken ?? "");
  }

  return (
    <AuthLayout>
      <form className="auth-card" onSubmit={submit}>
        <h1>Reset Password</h1>
        <p className="muted">Enter your admin email. In development, the reset token is shown here.</p>

        <label>
          Email
          <input value={email} onChange={(event) => setEmail(event.target.value)} />
        </label>

        <button className="primary-button">Send reset link</button>

        {message && <p className="success">{message}</p>}
        {token && <pre className="token-box">{token}</pre>}

        <button className="link-button" type="button" onClick={() => props.onNavigate("/login")}>
          Back to login
        </button>
      </form>
    </AuthLayout>
  );
}

function ResetPasswordPage(props: { onNavigate: (path: string) => void }) {
  const initialToken = new URLSearchParams(window.location.search).get("token") ?? "";
  const [token, setToken] = useState(initialToken);
  const [password, setPassword] = useState("Admin@123");
  const [message, setMessage] = useState("");

  async function submit(event: FormEvent) {
    event.preventDefault();

    const response = await fetch(`${API_BASE}/api/auth/reset-password`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ token, password })
    });

    const data = await response.json();
    setMessage(data.message);
  }
```

```
  return (
    <AuthLayout>
      <form className="auth-card" onSubmit={submit}>
        <h1>Create New Password</h1>

        <label>
          Reset token
          <input value={token} onChange={(event) => setToken(event.target.value)} />
        </label>

        <label>
          New password
          <input
            type="password"
            value={password}
            onChange={(event) => setPassword(event.target.value)}
          />
        </label>

        <button className="primary-button">Reset password</button>

        {message && <p className="success">{message}</p>}

        <button className="link-button" type="button" onClick={() => props.onNavigate("/login")}>
          Back to login
        </button>
      </form>
    </AuthLayout>
  );
}

function AuthLayout(props: { children: React.ReactNode }) {
  return (
    <main className="auth-page">
      <section className="auth-panel">
        <div>
          <p className="eyebrow">Campaign operations</p>
          <h2>Build, send, and measure email campaigns from one quiet workspace.</h2>
          <p>
            A focused admin console for teams managing contacts, templates, scheduled sends,
            compliance, and performance reporting.
          </p>
        </div>
      </section>
      {props.children}
    </main>
  );
}

function Shell(props: {
  auth: AuthState;
  route: string;
  children: React.ReactNode;
  onNavigate: (path: string) => void;
  onLogout: () => void;
}) {
  const visibleNav = navItems.filter((item) => item.roles.includes(props.auth.user.role));

  return (
    <main className="app-shell">
      <aside className="sidebar">
        <div className="sidebar-brand">
          <span className="brand-mark small">EC</span>
```

```
          <div>
            <strong>EmailOps</strong>
            <span>{props.auth.user.role.replace("_", " ")}</span>
          </div>
        </div>

        <nav>
          {visibleNav.map((item) => (
            <button
              key={item.path}
              className={props.route === item.path ? "nav-item active" : "nav-item"}
              onClick={() => props.onNavigate(item.path)}
            >
              {item.label}
            </button>
          ))}
        </nav>
      </aside>

      <section className="workspace">
        <header className="topbar">
          <div>
            <p className="eyebrow">Admin console</p>
            <h1>{getPageTitle(props.route)}</h1>
          </div>

          <div className="user-chip">
            <span>{props.auth.user.name ?? props.auth.user.email}</span>
            <button onClick={props.onLogout}>Logout</button>
          </div>
        </header>

        <section className="content">{props.children}</section>
      </section>
    </main>
  );
}

function DashboardPage(props: { user: User }) {
  const cards = [
    ["Total Contacts", "3", "+ seeded demo"],
    ["Emails Sent", "0", "ready for sending"],
    ["Open Rate", "0%", "tracking later"],
    ["Click Rate", "0%", "tracking later"]
  ];

  return (
    <>
      <div className="welcome-strip">
        <div>
          <h2>Welcome, {props.user.name ?? props.user.email}</h2>
          <p>M1 is active: authentication, roles, profile, and user management.</p>
        </div>
        <span className="status-pill">{props.user.role}</span>
      </div>

      <div className="metric-grid">
        {cards.map(([label, value, hint]) => (
          <article className="metric-card" key={label}>
            <span>{label}</span>
            <strong>{value}</strong>
            <small>{hint}</small>
          </article>
```

```
        ))}
      </div>

      <section className="panel">
        <h2>Recent Activity</h2>
        <div className="activity-row">Admin login and user role setup are ready.</div>
        <div className="activity-row">Contacts, templates, campaigns, and reports are protected
placeholders.</div>
      </section>
    </>
  );
}

function ProfilePage(props: { auth: AuthState; onAuthChange: (auth: AuthState) => void }) {
  const [name, setName] = useState(props.auth.user.name ?? "");
  const [email, setEmail] = useState(props.auth.user.email);
  const [currentPassword, setCurrentPassword] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [message, setMessage] = useState("");

  async function updateProfile(event: FormEvent) {
    event.preventDefault();

    const response = await fetch(`${API_BASE}/api/profile`, {
      method: "PUT",
      headers: authHeaders(props.auth.accessToken),
      body: JSON.stringify({ name, email })
    });

    const data = await response.json();

    if (response.ok) {
      props.onAuthChange({
        ...props.auth,
        user: data.user
      });
    }

    setMessage(response.ok ? "Profile updated" : data.message);
  }

  async function updatePassword(event: FormEvent) {
    event.preventDefault();

    const response = await fetch(`${API_BASE}/api/profile/password`, {
      method: "PUT",
      headers: authHeaders(props.auth.accessToken),
      body: JSON.stringify({ currentPassword, newPassword })
    });

    const data = await response.json();
    setMessage(response.ok ? "Password updated" : data.message);
  }

  return (
    <div className="two-column">
      <form className="panel form-panel" onSubmit={updateProfile}>
        <h2>Profile</h2>

        <label>
          Name
          <input value={name} onChange={(event) => setName(event.target.value)} />
        </label>
```

```
        <label>
          Email
          <input value={email} onChange={(event) => setEmail(event.target.value)} />
        </label>

        <button className="primary-button">Save profile</button>
      </form>

      <form className="panel form-panel" onSubmit={updatePassword}>
        <h2>Password</h2>

        <label>
          Current password
          <input
            type="password"
            value={currentPassword}
            onChange={(event) => setCurrentPassword(event.target.value)}
          />
        </label>

        <label>
          New password
          <input
            type="password"
            value={newPassword}
            onChange={(event) => setNewPassword(event.target.value)}
          />
        </label>

        <button className="primary-button">Update password</button>

        {message && <p className="success">{message}</p>}
      </form>
    </div>
  );
}

function UsersPage(props: { auth: AuthState }) {
  const [users, setUsers] = useState<User[]>([]);
  const [email, setEmail] = useState("newuser@example.com");
  const [name, setName] = useState("New User");
  const [role, setRole] = useState<Role>("CAMPAIGN_MANAGER");
  const [temporaryPassword, setTemporaryPassword] = useState("Temp@1234");
  const [message, setMessage] = useState("");

  async function loadUsers() {
    const response = await fetch(`${API_BASE}/api/users`, {
      headers: {
        Authorization: `Bearer ${props.auth.accessToken}`
      }
    });

    const data = await response.json();

    if (response.ok) {
      setUsers(data.users);
    } else {
      setMessage(data.message);
    }
  }

  async function inviteUser(event: FormEvent) {
```

```
    event.preventDefault();

    const response = await fetch(`${API_BASE}/api/users/invite`, {
      method: "POST",
      headers: authHeaders(props.auth.accessToken),
      body: JSON.stringify({ email, name, role, temporaryPassword })
    });

    const data = await response.json();
    setMessage(response.ok ? `Invited ${data.user.email}` : data.message);
    await loadUsers();
  }

  React.useEffect(() => {
    void loadUsers();
  }, []);

  return (
    <div className="two-column users-layout">
      <section className="panel">
        <div className="panel-header">
          <h2>Users</h2>
          <button onClick={loadUsers}>Refresh</button>
        </div>

        <div className="table">
          <div className="table-head">
            <span>Name</span>
            <span>Email</span>
            <span>Role</span>
          </div>
          {users.map((user) => (
            <div className="table-row" key={user.id}>
              <span>{user.name ?? "-"}</span>
              <span>{user.email}</span>
              <span>{user.role}</span>
            </div>
          ))}
        </div>
      </section>

      <form className="panel form-panel" onSubmit={inviteUser}>
        <h2>Invite User</h2>

        <label>
          Name
          <input value={name} onChange={(event) => setName(event.target.value)} />
        </label>

        <label>
          Email
          <input value={email} onChange={(event) => setEmail(event.target.value)} />
        </label>

        <label>
          Role
          <select value={role} onChange={(event) => setRole(event.target.value as Role)}>
            <option value="SUPER_ADMIN">Super Admin</option>
            <option value="CAMPAIGN_MANAGER">Campaign Manager</option>
            <option value="VIEWER">Viewer</option>
          </select>
        </label>
```

```
        <label>
          Temporary password
          <input
            value={temporaryPassword}
            onChange={(event) => setTemporaryPassword(event.target.value)}
          />
        </label>

        <button className="primary-button">Invite teammate</button>
        {message && <p className="success">{message}</p>}
      </form>
    </div>
  );
}

function PlaceholderPage(props: { route: string }) {
  return (
    <section className="panel">
      <h2>{getPageTitle(props.route)}</h2>
      <p className="muted">Protected placeholder screen. Full module implementation comes next.</p>
    </section>
  );
}

function getPageTitle(route: string) {
  const item = navItems.find((nav) => nav.path === route);

  if (item) {
    return item.label;
  }

  return "Workspace";
}

function authHeaders(token: string) {
  return {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`
  };
}

createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Part E: apps/web/src/styles.css

```
:root {
  font-family: Inter, "Segoe UI", Arial, sans-serif;
  color: #1f2937;
  background: #f4f6f8;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* {
  box-sizing: border-box;
}
```

```css
body {
  margin: 0;
}

button,
input,
select {
  font: inherit;
}

button {
  cursor: pointer;
}

.auth-page {
  min-height: 100vh;
  display: grid;
  grid-template-columns: minmax(320px, 1fr) 420px;
  background: #eef2f7;
}

.auth-panel {
  display: flex;
  align-items: flex-end;
  padding: 48px;
  color: #f8fafc;
  background:
    linear-gradient(rgba(17, 24, 39, 0.78), rgba(17, 24, 39, 0.78)),
    url("https://images.unsplash.com/photo-1497366754035-f200968a6e72?auto=format&fit=crop&w=1600&q=80");
  background-size: cover;
  background-position: center;
}

.auth-panel h2 {
  max-width: 680px;
  margin: 0 0 12px;
  font-size: 40px;
  line-height: 1.08;
}

.auth-panel p {
  max-width: 620px;
  color: #dbeafe;
}

.auth-card {
  align-self: center;
  width: min(100%, 420px);
  margin: auto;
  padding: 32px;
  background: #ffffff;
  border-left: 1px solid #d8dee9;
}

.auth-card h1 {
  margin: 18px 0 8px;
  font-size: 26px;
}

.brand-mark {
  width: 48px;
  height: 48px;
  display: grid;
```

```
  place-items: center;
  color: #ffffff;
  background: #155eef;
  border-radius: 8px;
  font-weight: 800;
}

.brand-mark.small {
  width: 36px;
  height: 36px;
  font-size: 13px;
}

label {
  display: grid;
  gap: 7px;
  margin: 16px 0;
  color: #4b5563;
  font-size: 14px;
  font-weight: 650;
}

input,
select {
  width: 100%;
  min-height: 42px;
  padding: 10px 12px;
  color: #111827;
  background: #ffffff;
  border: 1px solid #cfd7e3;
  border-radius: 6px;
  outline: none;
}

input:focus,
select:focus {
  border-color: #155eef;
  box-shadow: 0 0 0 3px rgba(21, 94, 239, 0.13);
}

.primary-button {
  min-height: 42px;
  padding: 10px 16px;
  color: #ffffff;
  background: #155eef;
  border: 0;
  border-radius: 6px;
  font-weight: 750;
}

.primary-button:disabled {
  opacity: 0.7;
}

.link-button {
  margin-top: 14px;
  padding: 0;
  color: #155eef;
  background: transparent;
  border: 0;
  font-weight: 650;
}
```

```
.error {
  padding: 10px 12px;
  color: #991b1b;
  background: #fee2e2;
  border-radius: 6px;
}

.success {
  padding: 10px 12px;
  color: #065f46;
  background: #d1fae5;
  border-radius: 6px;
}

.muted {
  color: #64748b;
}

.eyebrow {
  margin: 0 0 6px;
  color: #155eef;
  font-size: 12px;
  font-weight: 800;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}

.token-box {
  white-space: pre-wrap;
  overflow-wrap: anywhere;
  padding: 12px;
  color: #d1fae5;
  background: #111827;
  border-radius: 6px;
  font-size: 12px;
}

.app-shell {
  min-height: 100vh;
  display: grid;
  grid-template-columns: 260px 1fr;
}

.sidebar {
  position: sticky;
  top: 0;
  height: 100vh;
  padding: 18px;
  color: #cbd5e1;
  background: #111827;
}

.sidebar-brand {
  display: flex;
  gap: 10px;
  align-items: center;
  padding-bottom: 18px;
  border-bottom: 1px solid #263244;
}

.sidebar-brand strong {
  display: block;
  color: #ffffff;
```

```
}

.sidebar-brand span:last-child {
  font-size: 12px;
  color: #94a3b8;
}

.sidebar nav {
  display: grid;
  gap: 6px;
  margin-top: 18px;
}

.nav-item {
  min-height: 40px;
  padding: 10px 12px;
  color: #cbd5e1;
  text-align: left;
  background: transparent;
  border: 0;
  border-radius: 6px;
}

.nav-item:hover,
.nav-item.active {
  color: #ffffff;
  background: #1f2a3d;
}

.workspace {
  min-width: 0;
}

.topbar {
  min-height: 74px;
  display: flex;
  gap: 16px;
  align-items: center;
  justify-content: space-between;
  padding: 16px 24px;
  background: #ffffff;
  border-bottom: 1px solid #d8dee9;
}

.topbar h1 {
  margin: 0;
  font-size: 22px;
}

.user-chip {
  display: flex;
  gap: 10px;
  align-items: center;
}

.user-chip span {
  max-width: 260px;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.user-chip button,
```

```css
.panel-header button {
  min-height: 36px;
  padding: 8px 12px;
  color: #334155;
  background: #f8fafc;
  border: 1px solid #cfd7e3;
  border-radius: 6px;
}

.content {
  padding: 24px;
}

.welcome-strip,
.panel,
.metric-card {
  background: #ffffff;
  border: 1px solid #d8dee9;
  border-radius: 8px;
}

.welcome-strip {
  display: flex;
  gap: 16px;
  align-items: center;
  justify-content: space-between;
  padding: 22px;
}

.welcome-strip h2 {
  margin: 0 0 4px;
}

.welcome-strip p {
  margin: 0;
  color: #64748b;
}

.status-pill {
  padding: 7px 10px;
  color: #155eef;
  background: #e0ecff;
  border-radius: 999px;
  font-size: 12px;
  font-weight: 800;
}

.metric-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
  margin: 16px 0;
}

.metric-card {
  padding: 18px;
}

.metric-card span {
  display: block;
  color: #64748b;
  font-size: 13px;
}
```

```
.metric-card strong {
  display: block;
  margin: 8px 0 4px;
  color: #111827;
  font-size: 28px;
}

.metric-card small {
  color: #64748b;
}

.panel {
  padding: 20px;
}

.panel h2 {
  margin: 0 0 14px;
}

.activity-row {
  padding: 12px 0;
  border-top: 1px solid #e5e7eb;
}

.two-column {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}

.users-layout {
  grid-template-columns: 1.4fr 0.8fr;
}

.form-panel {
  align-self: start;
}

.panel-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.table {
  overflow: auto;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
}

.table-head,
.table-row {
  display: grid;
  grid-template-columns: 1fr 1.5fr 1fr;
  min-width: 620px;
}

.table-head {
  color: #475569;
  background: #f8fafc;
  font-size: 12px;
  font-weight: 800;
```

```
  text-transform: uppercase;
}

.table-head span,
.table-row span {
  padding: 12px;
  border-bottom: 1px solid #e5e7eb;
}

.table-row:last-child span {
  border-bottom: 0;
}

@media (max-width: 900px) {
  .auth-page {
    grid-template-columns: 1fr;
  }

  .auth-panel {
    min-height: 280px;
    padding: 28px;
  }

  .auth-panel h2 {
    font-size: 30px;
  }

  .auth-card {
    width: 100%;
    min-height: auto;
    border-left: 0;
  }

  .app-shell {
    grid-template-columns: 1fr;
  }

  .sidebar {
    position: static;
    height: auto;
  }

  .sidebar nav {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .topbar {
    align-items: flex-start;
    flex-direction: column;
  }

  .metric-grid,
  .two-column,
  .users-layout {
    grid-template-columns: 1fr;
  }

  .content {
    padding: 16px;
  }
}

@media (max-width: 520px) {
```

```
  .sidebar nav {
    grid-template-columns: 1fr;
  }

  .welcome-strip,
  .user-chip {
    align-items: flex-start;
    flex-direction: column;
  }

  .auth-card {
    padding: 24px 18px;
  }

  .metric-card strong {
    font-size: 24px;
  }
}
```